Pontificia Universidad Javeriana

Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

Bases de Datos: Administración y Aplicaciones (Período 2026-3)

PROYECTO RESTAURANTE

Diseño Relacional, Almacenamiento Físico, Integridad Avanzada y Consultas Analíticas en PostgreSQL 18

Presentado por:

Estudiante(s):
Maria Alejandra García & Samuel Isaias Pico
Ingeniería de Sistemas

Docente Titular:

Ing. Ricardo Naranjo Faccini, MSc.
Profesor Asistente
Departamento de Ingeniería de Sistemas

Bogotá D.C., Colombia
Noviembre de 2026

Resumen Ejecutivo

El presente documento contiene el informe técnico y formal de entrega correspondiente al **Proyecto Restaurante** para la asignatura de *Bases de Datos: Administración y Aplicaciones* de la Pontificia Universidad Javeriana.

El objetivo primordial consiste en diseñar, implementar y documentar una base de datos relacional robusta en **PostgreSQL 18** para soportar la operación integral de un restaurante de alta demanda. El sistema gestiona mesas, reservaciones, menús, pedidos y entregas, garantizando reglas de negocio críticas: control estricto del cupo total de mesas, prevención matemática de cruces de reservaciones en ventanas mínimas de dos horas, despacho de platos hacia cocineros autorizados según especialidades capacitadas, y trazabilidad completa del ciclo de vida de los pedidos para los meseros y maitres.

Asimismo, se expone la arquitectura física basada en **tablespaces** en directorios independientes de almacenamiento, particionamiento de tablas por rangos y fechas, disparadores (*triggers*) de integridad referencial activa, y el catálogo exhaustivo de consultas SQL optimizadas para resolver los casos de uso operativos y reportes analíticos de gestión.

Índice

1. Descripción General y Roles de Usuario **1**
- 1.1. Contexto y Objetivos del Sistema . . . 1
- 1.2. Matriz de Roles y Responsabilidades . . . 1

2. Casos de Uso del Sistema **2**

3. Diseño de la Base de Datos y Modelo Entidad-Relación **3**
- 3.1. Modelo Entidad-Relación Obligatorio . . . 3
- 3.2. Normalización del Esquema . . . 3

4. Diccionario de Datos Exhaustivo **4**
- 4.1. Módulo de Usuarios, Seguridad y Roles . . . 4
- 4.2. Módulo de Menú y Especialidades . . . 4
- 4.3. Módulo de Mesas, Reservaciones y Horarios . . . 5
- 4.4. Módulo de Pedidos, Órdenes y Preparaciones . . . 5

5. Arquitectura Física: Tablespaces y Particionamiento **6**
- 5.1. Configuración de Tablespaces en PostgreSQL . . . 6
- 5.2. Estrategia de Particionamiento de Tablas . . . 6

6. Integridad de Datos y Disparadores (Triggers) **8**
- 6.1. Trigger 1: Prevención de Cruces de Reservaciones (2 Horas) . . . 8
- 6.2. Trigger 2: Validación Estricta de Cupo Total del Restaurante . . . 9
- 6.3. Trigger 3: Acreditación de Especialidad al Cocinero . . . 9

7. Consultas SQL de los Casos de Uso y Reportes **11**
- 7.1. CU-01: Cupo Total del Restaurante y Aforo . . . 11
- 7.2. CU-02: Disponibilidad y Alerta de Reservas para el Maitre . . . 11

7.3. CU-05: Cola de Cocina Priorizada por Tiempo Descendente 11
7.4. CU-07: Reportes Estadísticos Analíticos 12

8. Estrategia de Pruebas y Validación 13
8.1. Conjunto de Datos de Prueba Determinista 13
8.2. Pruebas Automatizadas de Integración 13

Índice de tablas

1. Matriz de roles y permisos del sistema del restaurante. 1
2. Diccionario de datos: Usuarios, Roles y Actuaciones. 4
3. Diccionario de datos: Menú y Especialidades culinarias. 4
4. Diccionario de datos: Mesas, Reservaciones y Horarios. 5
5. Diccionario de datos: Pedidos, Órdenes y Preparaciones. 5

1. Descripción General y Roles de Usuario

1.1. Contexto y Objetivos del Sistema

El proyecto aborda la problemática operacional de establecimientos gastronómicos en donde confluyen simultáneamente múltiples perfiles de usuarios y flujos transaccionales altamente concurrentes. El modelo debe garantizar:

- **Aforo y Asignación de Mesas:** No exceder en ningún momento el cupo máximo de comensales del local ni solapar reservas sobre un mismo recurso físico.
- **Gestión de Horarios y Reservaciones:** Todo turno de reserva estima una ocupación predeterminada de dos (2) horas, calculando la disponibilidad a través de álgebra de intervalos (`tsrange`).
- **Especialización en Cocina:** Cada cocinero solo puede visualizar, tomar y marcar como preparados aquellos platos para los cuales ha sido formalmente acreditado mediante una especialidad.
- **Gestión Operativa de Pedidos:** Los meseros registran órdenes asociadas a mesas y clientes, gestionando la transición de estados (*Solicitada* $\rightarrow$ *En preparación* $\rightarrow$ *Preparada* $\rightarrow$ *Entregada*).
- **Inteligencia de Negocio:** Generación dinámica de reportes consolidados por día, semana y mes para ventas, afluencia de reservas y platos de mayor demanda.

1.2. Matriz de Roles y Responsabilidades

Rol	Alcance Operativo	Acciones Principales en Base de Datos
Administrador	Control maestro de infraestructura, personal y finanzas	Mantenimiento de mesas (CRUD), actualización de carta/precios, asignación de roles y especialidades a empleados, y consulta de reportes analíticos.
Maitre	Control de sala, aforo y relaciones de acceso	Registro, consulta y cancelación de reservaciones; verificación de cupo total en tiempo real; atención de alertas tempranas (≤ 2 horas).
Mesero	Toma de comandas y servicio a mesa	Creación de pedidos con múltiples ítems, consulta de comandas listas en cocina y registro de entrega efectiva en mesa.
Cocinero	Elaboración culinaria en cocina	Visualización de cola de comandas autorizadas priorizadas por tiempo descendente de cocción, toma de órdenes y notificación de plato listo.
Cliente	Comensal (interacción indirecta)	Titular de reservaciones y pedidos; consulta de historial de consumos y visitas.

Tabla 1: Matriz de roles y permisos del sistema del restaurante.

2. Casos de Uso del Sistema

A continuación se detallan los siete casos de uso troncales y las funcionalidades transversales estipuladas en la especificación oficial:

CU-1: Gestión de Mesas (Administrador): Creación, actualización y retiro de mesas físicas, asignando su número único y número de asientos disponibles. Cálculo automático del aforo o cupo total mediante $\sum$ sillas.

CU-2: Gestión de Reservaciones (Maitre): Asignación de mesa y horario a clientes. Se exige validar que la mesa seleccionada soporte el número de comensales, que no existan traslapes temporales con otras reservas en dicha mesa (considerando duración estándar de 2 horas), y que la suma de asistentes concurrentes no sobrepase el cupo del restaurante.

CU-3: Gestión del Menú (Administrador): Catálogo de ítems clasificados en cuatro categorías obligatorias (*Entrada, Plato Fuerte, Postre, Bebida*). Cada plato registra nombre, descripción textual, precio unitario vigente y tiempo estimado de preparación (`INTERVAL`). Registro de especialidades por cocinero.

CU-4: Registro de Pedidos (Mesero): Creación de pedidos cabecera vinculados a una mesa y un cliente. Inclusión de una o múltiples líneas de orden (`ordenes`) capturando el precio unitario vigente en ese instante. Estado inicial automático en *Solicitado/En preparación*.

CU-5: Preparación de Platos (Cocinero): Vista en cocina donde cada cocinero accede únicamente a las órdenes de platos que está capacitado para elaborar. Las órdenes se ordenan de forma descendente por tiempo de cocción (*los platos más demorados de primero*) y secundariamente por antigüedad de solicitud. Al culminar, actualiza el estado a *Preparado*.

CU-6: Gestión de Entregas (Mesero): Monitoreo de ítems con estado *Preparado* listos para servir. El mesero efectúa la entrega material a la mesa y actualiza el estado del ítem a *Entregado*.

CU-7: Reportes y Estadísticas (Administrador): Consultas analíticas agrupadas dinámicamente por día, semana o mes (`date_trunc`) que consolidan:

- Volumen y estados de reservaciones.
- Ranking de los platos más solicitados (unidades e ingresos).
- Facturación global y ventas verificadas por fecha de entrega.

Funcionalidades Adicionales Implementadas:

- **Sistema de Alertas y Notificaciones:** Alertas automáticas para el Maitre ante reservas que inician en ≤ 2 horas, y para el Mesero ante órdenes listas para entrega.
- **Historial Completo de Clientes:** Métrica consolidada por cliente con visitas históricas, pedidos consumidos y valor monetario acumulado.
- **Gestión Integral de Personal:** Control de altas, bajas lógicas (`activo`) y modificación de roles múltiples mediante la entidad asociativa `actuaciones`.

3. Diseño de la Base de Datos y Modelo Entidad-Relación

3.1. Modelo Entidad-Relación Obligatorio

El sistema implementa de manera estricta el esquema relacional de 12 entidades especificado por la cátedra. La Figura 1 ilustra la topología relacional y las dependencias foráneas.

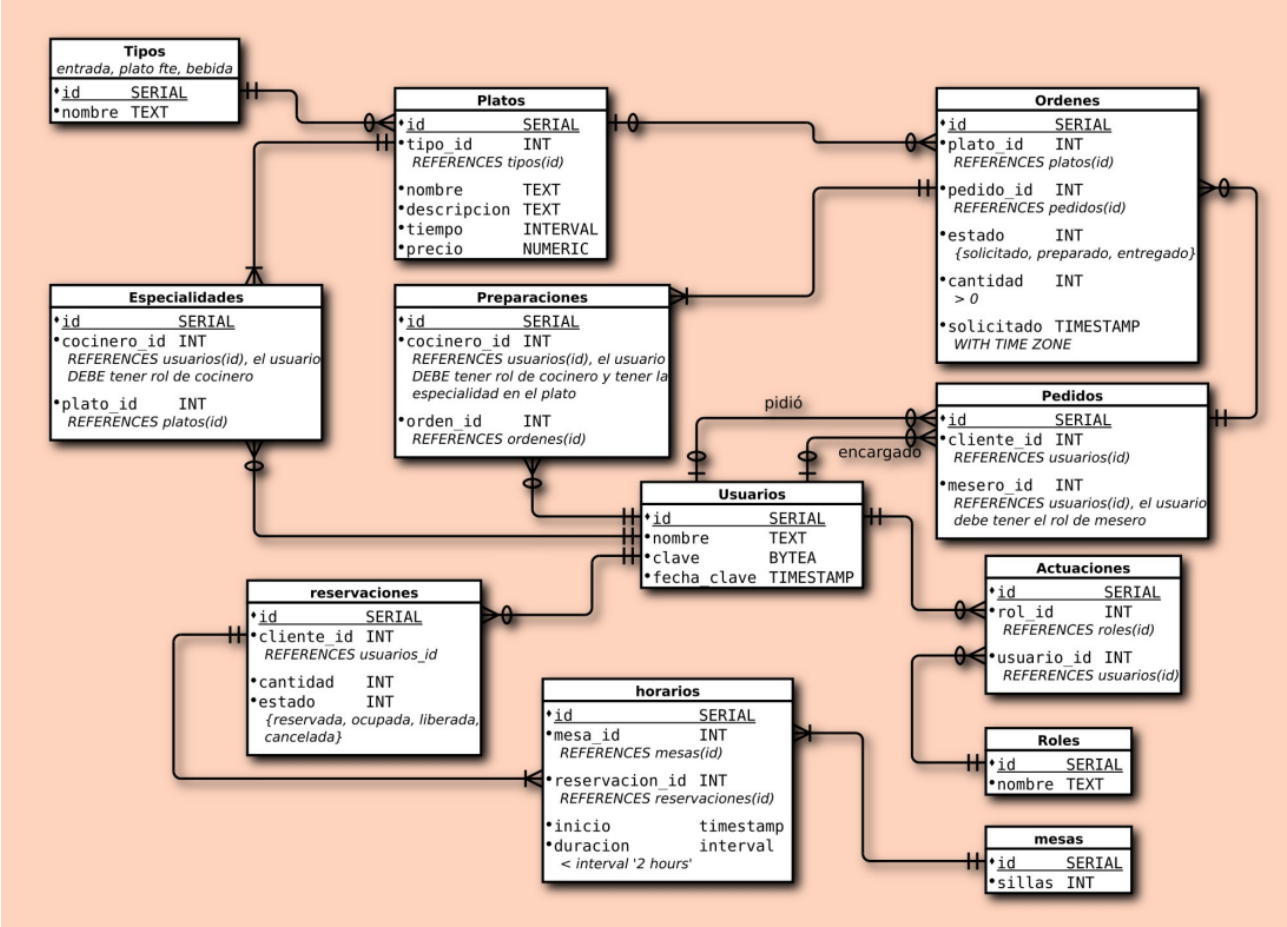


Figura 1: Diagrama E-R

3.2. Normalización del Esquema

El diseño satisface la **Tercera Forma Normal (3FN)**:

1. **1FN:** Todos los atributos son atómicos y no existen grupos repetitivos. Los roles múltiples de los usuarios se resuelven a través de la tabla asociativa `actuaciones`.

2. **2FN:** Toda columna que no forma parte de una clave candidata depende por completo de la clave primaria completa (claves subrogadas simples mediante secuencias `SERIAL`).

3. **3FN:** No existen dependencias transitivas entre atributos no clave. Por ejemplo, el precio y categoría no se embeben en la orden, sino que se normalizan en `platos` y `tipos`, guardando en la orden únicamente el snapshot transaccional (`precio_unitario`).

4. Diccionario de Datos Exhaustivo

A continuación se documenta la estructura de campos, tipos nativos de PostgreSQL 18, claves y restricciones de integridad para las 12 entidades del sistema:

4.1. Módulo de Usuarios, Seguridad y Roles

Columna	Tipo Dato	Nulo	Claves / Constraints	Descripción Funcional
Tabla: public.usuarios (Usuarios del sistema: clientes, cocineros, meseros, maitres, admin)				
id	SERIAL	No	PRIMARY KEY	Identificador único subrogado del usuario.
nombre	TEXT	No	–	Nombre completo del usuario o razón social.
clave	BYTEA	No	–	Hash criptográfico de autenticación (SHA-256).
fecha_clave	TIMESTAMP	No	DEFAULT now()	Marca temporal del último cambio de contraseña.
activo	BOOLEAN	No	DEFAULT true	Estado de habilitación operativa para login.
Tabla: public.roles (Catálogo de roles del restaurante)				
id	SERIAL	No	PRIMARY KEY	Identificador único del rol.
nombre	TEXT	No	UNIQUE	Nombre del rol: Administrador, Maitre, Mesero, Cocinero, Cliente.
Tabla: public.actuaciones (Asociación N:M entre usuarios y roles)				
id	SERIAL	No	PRIMARY KEY	Identificador de asignación de rol.
rol_id	INTEGER	No	FK ->roles(id)	Rol concedido al usuario.
usuario_id	INTEGER	No	FK ->usuarios(id)	Usuario al cual se le otorga el rol.

Tabla 2: Diccionario de datos: Usuarios, Roles y Actuaciones.

4.2. Módulo de Menú y Especialidades

Columna	Tipo Dato	Nulo	Claves / Constraints	Descripción Funcional
Tabla: public.tipos (Categorías del menú)				
id	SERIAL	No	PRIMARY KEY	Identificador de categoría.
nombre	TEXT	No	UNIQUE	Entrada, Plato Fuerte, Postre o Bebida.
Tabla: public.platos (Catálogo de platos y bebidas ofrecidos)				
id	SERIAL	No	PRIMARY KEY	Identificador del plato.
tipo_id	INTEGER	No	FK ->tipos(id)	Categoría a la que pertenece el ítem.
nombre	TEXT	No	–	Nombre comercial del plato o bebida.
descripcion	TEXT	Sí	–	Reseña de ingredientes y presentación.
tiempo	INTERVAL	No	CHECK (>'0')	Duración promedio estimada de cocción/preparación.
precio	NUMERIC	No	CHECK (precio >= 0)	Precio de venta en moneda local (COP).
activo	BOOLEAN	No	DEFAULT true	Habilitado en la carta para nuevos pedidos.
Tabla: public.especialidades (Autorización de platos para cada cocinero)				
id	SERIAL	No	PRIMARY KEY	Identificador de acreditación.
cocinero_id	INTEGER	No	FK ->usuarios(id)	Usuario con rol Cocinero autorizado.
plato_id	INTEGER	No	FK ->platos(id)	Plato que el cocinero sabe elaborar.

Tabla 3: Diccionario de datos: Menú y Especialidades culinarias.

4.3. Módulo de Mesas, Reservaciones y Horarios

Columna	Tipo Dato	Nulo	Claves / Constraints	Descripción Funcional
Tabla: public.mesas (Mesas físicas del establecimiento)				
id	SERIAL	No	PRIMARY KEY	Identificador o número físico de la mesa.
sillas	INTEGER	No	CHECK (sillas >0)	Número de puestos o capacidad de comensales.
Tabla: public.reservaciones (Tabla particionada de reservaciones solicitadas)				
id	INTEGER	No	PRIMARY KEY	Identificador de la reservación.
cliente_id	INTEGER	No	FK ->usuarios(id)	Cliente titular que realiza la reservación.
cantidad	INTEGER	No	CHECK (cantidad >0)	Número de personas que asistirán.
estado	INTEGER	No	CHECK (1,2,3,4)	1=Pendiente, 2=Confirmada, 3=Completada, 4=Cancelada.
Tabla: public.horarios (Asignación temporal de mesa a cada reservación)				
id	SERIAL	No	PRIMARY KEY	Identificador de agenda.
mesa_id	INTEGER	No	FK ->mesas(id)	Mesa asignada a la reserva.
reservacion_id	INTEGER	No	FK ->reservaciones	Reservación vinculada.
inicio	TIMESTAMP	No	–	Fecha y hora programada de inicio del turno.
duracion	INTERVAL	No	CHECK (<= '2 hours')	Duración estimada (fijada en 2 horas).

Tabla 4: Diccionario de datos: Mesas, Reservaciones y Horarios.

4.4. Módulo de Pedidos, Órdenes y Preparaciones

Columna	Tipo Dato	Nulo	Claves / Constraints	Descripción Funcional
Tabla: public.pedidos (Cabecera de comanda en mesa)				
id	SERIAL	No	PRIMARY KEY	Identificador del pedido.
cliente_id	INTEGER	No	FK ->usuarios(id)	Cliente que realiza el consumo.
mesero_id	INTEGER	No	FK ->usuarios(id)	Empleado con rol Mesero que atiende.
mesa_id	INTEGER	Sí	FK ->mesas(id)	Mesa donde se sirve el pedido.
reservacion_id	INTEGER	Sí	FK ->reservaciones	Reserva asociada si el pedido deriva de ella.
creado	TIMESTAMPTZ	No	DEFAULT now()	Fecha y hora exacta de apertura del pedido.
token	TEXT	Sí	UNIQUE	Token de idempotencia contra reenvíos HTTP.
Tabla: public.ordenes (Detalle de platos solicitados en un pedido)				
id	SERIAL	No	PRIMARY KEY	Identificador de línea de pedido.
plato_id	INTEGER	No	FK ->platos(id)	Ítem culinario ordenado.
pedido_id	INTEGER	No	FK ->pedidos(id)	Pedido cabecera al que pertenece.
estado	INTEGER	No	CHECK (1,2,3,4)	1=Solicitada, 2=En preparación, 4=Preparada, 3=Entregada.
cantidad	INTEGER	No	CHECK (cantidad >0)	Número de porciones solicitadas.
solicitado	TIMESTAMPTZ	No	DEFAULT now()	Marca temporal del envío a cocina.
precio_unitario	NUMERIC	Sí	CHECK (>= 0)	Precio congelado al momento del registro.
preparado	TIMESTAMPTZ	Sí	–	Instante en que cocina finaliza el plato.
entregado	TIMESTAMPTZ	Sí	–	Instante en que el mesero sirve en mesa.
Tabla: public.preparaciones (Asignación de cocinero a una orden)				
id	SERIAL	No	PRIMARY KEY	Identificador de preparación.
cocinero_id	INTEGER	No	FK ->usuarios(id)	Cocinero acreditado que elabora el plato.
orden_id	INTEGER	No	FK ->ordenes(id)	Orden específica asignada.

Tabla 5: Diccionario de datos: Pedidos, Órdenes y Preparaciones.

5. Arquitectura Física: Tablespaces y Particionamiento

5.1. Configuración de Tablespaces en PostgreSQL

Conforme al Requisito Técnico 6.1, la base de datos aprovecha múltiples espacios de tablas (*tablespaces*) ubicados en rutas independientes del disco duro para segregar físicamente las cargas de entrada/salida (I/O). En el clúster se encuentran configurados los tablespaces de reservaciones:

- `ts_res_1`: Directorio físico de almacenamiento para el primer tramo de reservaciones.
- `ts_res_2`: Directorio físico de almacenamiento para el segundo tramo de reservaciones.
- `ts_res_3`: Directorio físico de almacenamiento para el tercer tramo de reservaciones históricas.

Esta arquitectura permite separar los discos mecánicos o volúmenes SSD, balanceando la concurrencia entre transacciones activas de lectura y escrituras analíticas.

5.2. Estrategia de Particionamiento de Tablas

La tabla `reservaciones` se encuentra particionada por rangos (`PARTITION BY RANGE (id)`), distribuyendo sus registros en particiones hijas alojadas en los tablespaces independientes:

```
-- Tabla principal declarativa particionada
CREATE TABLE public.reservaciones (
    id integer NOT NULL,
    cliente_id integer NOT NULL,
    cantidad integer NOT NULL,
    estado integer NOT NULL,
    CONSTRAINT ck_reservaciones_cantidad_positiva CHECK (cantidad > 0),
    CONSTRAINT ck_reservaciones_estado CHECK (estado IN (1, 2, 3, 4))
) PARTITION BY RANGE (id);

-- Asignaci n de particiones a sus respectivos tablespaces
CREATE TABLE public.reservaciones_1 PARTITION OF public.reservaciones
    FOR VALUES FROM (1) TO (15001) TABLESPACE ts_res_1;

CREATE TABLE public.reservaciones_2 PARTITION OF public.reservaciones
    FOR VALUES FROM (15001) TO (30001) TABLESPACE ts_res_2;

CREATE TABLE public.reservaciones_3 PARTITION OF public.reservaciones
    FOR VALUES FROM (30001) TO (45001) TABLESPACE ts_res_3;

-- Partici n por defecto para cubrir crecimientos futuros sin bloqueo
CREATE TABLE public.reservaciones_futuras PARTITION OF public.reservaciones DEFAULT;
```

Particionamiento por Fechas (Diseño de Eventos): Para tablas de series temporales de alta cardinalidad como `ordenes` y `pedidos`, el particionamiento idóneo sobre los tablespaces definidos se formula mediante rangos de fechas (`RANGE (solicitado)`), permitiendo la exclusión de particiones (*Partition Pruning*) en consultas mensuales o anuales:

```
-- Ejemplo de particionamiento temporal por meses para ventas
CREATE TABLE public.ordenes_part (
    id SERIAL,
    solicitado TIMESTAMPTZ NOT NULL,
    ...
) PARTITION BY RANGE (solicitado);
```

```
7
8  CREATE TABLE public.ordenes_2026_q3 PARTITION OF public.ordenes_part
9      FOR VALUES FROM ('2026-07-01') TO ('2026-10-01') TABLESPACE ts_res_2;
10
11 CREATE TABLE public.ordenes_2026_q4 PARTITION OF public.ordenes_part
12     FOR VALUES FROM ('2026-10-01') TO ('2027-01-01') TABLESPACE ts_res_1;
```

6. Integridad de Datos y Disparadores (Triggers)

El Requisito Técnico 6.3 exige la implementación de mecanismos de integridad relacional activa en el motor de base de datos. En el script `01_integridad_y_triggers.sql` se implementaron cinco disparadores (*triggers*) clave en PL/pgSQL:

6.1. Trigger 1: Prevención de Cruces de Reservaciones (2 Horas)

Evita matemáticamente que se solapen dos reservaciones no canceladas en la misma mesa durante un bloque mínimo de dos horas:

```
CREATE OR REPLACE FUNCTION public.fn_trg_validar_cruce_reserva()
RETURNS TRIGGER AS $$
DECLARE
    v_capacidad_mesa integer;
    v_personas_reserva integer;
    v_estado_reserva integer;
    v_conflicto_id integer;
    v_duracion interval;
BEGIN
    SELECT cantidad, estado INTO v_personas_reserva, v_estado_reserva
    FROM public.reservaciones WHERE id = NEW.reservacion_id;

    IF v_estado_reserva = 4 THEN RETURN NEW; END IF; -- Cancelada

    SELECT sillas INTO v_capacidad_mesa FROM public.mesas WHERE id = NEW.mesa_id;
    IF v_personas_reserva > v_capacidad_mesa THEN
        RAISE EXCEPTION 'Capacidad excedida: Reserva para % personas en mesa de % sillas.',
            v_personas_reserva, v_capacidad_mesa;
    END IF;

    v_duracion := GREATEST(COALESCE(NEW.duracion, interval '2 hours'), interval '2 hours');

    SELECT h.id INTO v_conflicto_id
    FROM public.horarios h
    JOIN public.reservaciones r ON r.id = h.reservacion_id
    WHERE h.mesa_id = NEW.mesa_id AND r.estado <> 4
      AND h.id <> COALESCE(NEW.id, -1) AND h.reservacion_id <> NEW.reservacion_id
      AND tsrange(h.inicio, h.inicio + GREATEST(h.duracion, interval '2 hours'), '[)')
       && tsrange(NEW.inicio, NEW.inicio + v_duracion, '[)')
    LIMIT 1;

    IF v_conflicto_id IS NOT NULL THEN
        RAISE EXCEPTION 'Cruce de horario: Mesa #% ya reservada (conflicto #%).',
            NEW.mesa_id, v_conflicto_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_cruce_reserva
BEFORE INSERT OR UPDATE OF mesa_id, inicio, duracion ON public.horarios
FOR EACH ROW EXECUTE FUNCTION public.fn_trg_validar_cruce_reserva();
```

6.2. Trigger 2: Validación Estricta de Cupo Total del Restaurante

Garantiza que la sumatoria de comensales concurrentes en la ventana horaria no supere el cupo total del restaurante ($\sum$ sillas):

```
CREATE OR REPLACE FUNCTION public.fn_trg_validar_cupo_restaurante()
RETURNS TRIGGER AS $$
DECLARE
    v_cupo_total integer;
    v_ocupacion_pico integer;
    v_personas_nuevas integer;
BEGIN
    SELECT cantidad INTO v_personas_nuevas FROM public.reservaciones WHERE id =
        NEW.reservacion_id;
    SELECT COALESCE(SUM(sillas), 0) INTO v_cupo_total FROM public.mesas;

    WITH ocupaciones AS (
        SELECT h.inicio, h.inicio + GREATEST(h.duracion, interval '2 hours') AS fin,
            r.cantidad
        FROM public.horarios h JOIN public.reservaciones r ON r.id = h.reservacion_id
        WHERE r.estado <> 4 AND h.reservacion_id <> NEW.reservacion_id
          AND h.inicio < NEW.inicio + interval '2 hours'
          AND h.inicio + GREATEST(h.duracion, interval '2 hours') > NEW.inicio
    ),
    puntos AS (
        SELECT NEW.inicio AS t UNION SELECT inicio FROM ocupaciones WHERE inicio >=
            NEW.inicio
    )
    SELECT COALESCE(MAX((SELECT COALESCE(SUM(cantidad),0) FROM ocupaciones o WHERE
        o.inicio<=p.t AND o.fin>p.t)),0)
    INTO v_ocupacion_pico FROM puntos p;

    IF (v_ocupacion_pico + v_personas_nuevas) > v_cupo_total THEN
        RAISE EXCEPTION 'Cupo total superado: Capacidad=%, concurrencia
            solicitada=%.',
            v_cupo_total, (v_ocupacion_pico + v_personas_nuevas);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_cupo_restaurante
BEFORE INSERT OR UPDATE OF inicio, duracion ON public.horarios
FOR EACH ROW EXECUTE FUNCTION public.fn_trg_validar_cupo_restaurante();
```

6.3. Trigger 3: Acreditación de Especialidad al Cocinero

Comprueba que el cocinero que toma una orden posea el rol 'Cocinero' y tenga registrada la especialidad del plato correspondiente:

```
CREATE OR REPLACE FUNCTION public.fn_trg_validar_preparacion_cocinero()
RETURNS TRIGGER AS $$
DECLARE
    v_es_cocinero boolean;
    v_tiene_especialidad boolean;
    v_plato_id integer;
BEGIN
    SELECT EXISTS (
```

```
        SELECT 1 FROM public.actuaciones a JOIN public.roles r ON r.id = a.rol_id
        WHERE a.usuario_id = NEW.cocinero_id AND r.nombre = 'Cocinero'
    ) INTO v_es_cocinero;

    IF NOT v_es_cocinero THEN
        RAISE EXCEPTION 'El usuario #% no tiene el rol de Cocinero.',
            NEW.cocinero_id;
    END IF;

    SELECT plato_id INTO v_plato_id FROM public.ordenes WHERE id = NEW.orden_id;

    SELECT EXISTS (
        SELECT 1 FROM public.especialidades
        WHERE cocinero_id = NEW.cocinero_id AND plato_id = v_plato_id
    ) INTO v_tiene_especialidad;

    IF NOT v_tiene_especialidad THEN
        RAISE EXCEPTION 'Especialidad no autorizada: Cocinero #% no autorizado para
            plato #%.',
            NEW.cocinero_id, v_plato_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_preparacion_cocinero
BEFORE INSERT OR UPDATE OF cocinero_id, orden_id ON public.preparaciones
FOR EACH ROW EXECUTE FUNCTION public.fn_trg_validar_preparacion_cocinero();
```

7. Consultas SQL de los Casos de Uso y Reportes

El script `02_consultas_casos_de_uso.sql` consolida las consultas relacionales requeridas. A continuación se presentan las más representativas:

7.1. CU-01: Cupo Total del Restaurante y Aforo

```sql
SELECT
    count(id)               AS total_mesas,
    sum(sillas)             AS cupo_total_restaurante,
    min(sillas)             AS capacidad_minima_mesa,
    max(sillas)             AS capacidad_maxima_mesa,
    round(avg(sillas), 2)   AS promedio_sillas_por_mesa
FROM public.mesas;
```

7.2. CU-02: Disponibilidad y Alerta de Reservas para el Maitre

```sql
-- Alertas de reservas proximas a comenzar en ventana <= 2 horas
SELECT
    r.id AS reservacion_id,
    u.nombre AS cliente,
    h.mesa_id,
    h.inicio,
    (h.inicio - LOCALTIMESTAMP) AS tiempo_restante,
    r.cantidad AS personas
FROM public.reservaciones r
JOIN public.usuarios u ON u.id = r.cliente_id
JOIN public.horarios h ON h.reservacion_id = r.id
WHERE r.estado IN (1, 2)
  AND h.inicio >= LOCALTIMESTAMP
  AND h.inicio <= LOCALTIMESTAMP + interval '2 hours'
ORDER BY h.inicio ASC;
```

7.3. CU-05: Cola de Cocina Priorizada por Tiempo Descendente

El cocinero consulta únicamente las órdenes pendientes que sabe cocinar, priorizando los platos más demorados de primero:

```sql
SELECT
    o.id AS orden_id,
    ped.mesa_id,
    p.nombre AS plato,
    p.tiempo AS tiempo_preparacion,
    o.cantidad,
    o.solicitado
FROM public.ordenes o
JOIN public.platos p ON p.id = o.plato_id
JOIN public.pedidos ped ON ped.id = o.pedido_id
JOIN public.especialidades esp ON esp.plato_id = p.id AND esp.cocinero_id =
    :p_cocinero_id
WHERE o.estado = 1 -- Solicitadas y disponibles
ORDER BY p.tiempo DESC, o.solicitado ASC;
```

7.4. CU-07: Reportes Estadísticos Analíticos

```sql
-- Reporte de Ventas por periodo mensual (date_trunc)
SELECT
    date_trunc('month', COALESCE(o.entregado, o.solicitado) AT TIME ZONE
        'America/Bogota')::date AS periodo,
    count(DISTINCT o.pedido_id) AS total_pedidos,
    sum(o.cantidad) AS platos_servidos,
    sum(o.cantidad * o.precio_unitario) FILTER (WHERE o.entregado IS NOT NULL) AS
        ventas_verificadas,
    sum(o.cantidad * COALESCE(o.precio_unitario, p.precio)) AS facturacion_total
FROM public.ordenes o
JOIN public.platos p ON p.id = o.plato_id
WHERE o.estado = 3 -- Entregado
GROUP BY 1 ORDER BY 1;

-- Ranking Top de Platos mas solicitados
SELECT
    p.nombre AS plato,
    t.nombre AS categoria,
    sum(o.cantidad) AS unidades_vendidas,
    sum(o.cantidad * COALESCE(o.precio_unitario, p.precio)) AS facturacion
FROM public.ordenes o
JOIN public.platos p ON p.id = o.plato_id
JOIN public.tipos t ON t.id = p.tipo_id
GROUP BY p.id, p.nombre, t.nombre
ORDER BY unidades_vendidas DESC LIMIT 10;
```

8. Estrategia de Pruebas y Validación

8.1. Conjunto de Datos de Prueba Determinista

Para garantizar la reproducibilidad científica de las evaluaciones, se utilizó la semilla determinista `03_datos_prueba_Garcia_Pico.sql` con factor `data_seed = 42`. Esta semilla genera:

- 32 mesas con configuraciones variadas de 2 a 8 sillas.
- Plantilla completa de empleados (4 Maitres, 12 Cocineros con capacitación cruzada, 10 Meseros y 2 Administradores).
- Catálogo completo de 24 platos clasificados en Entradas, Fuertes, Postres y Bebidas.
- 10.000 clientes sintéticos con más de 18.000 reservaciones y órdenes históricas.

8.2. Pruebas Automatizadas de Integración

La suite de pruebas disponible en `tests/integration.php` valida exhaustivamente:

1. **Aislamiento de Entorno:** Restricción estricta sobre la base de pruebas `restaurante_test` para impedir mutación accidental de datos en producción.
2. **Idempotencia:** Uso de advisory locks (`pg_advisory_xact_lock`) y tokens de un solo uso que rechazan duplicación de pedidos ante reenvíos concurrentes.
3. **Control de Estados y Privilegios:** Imposibilidad de que un mesero entregue un pedido antes de ser preparado, o que un cocinero sin especialidad tome una comanda.